

Chapter 4

Worksheet

Instructions:

Please complete all activities in this worksheet. Write your answers in the spaces provided or on separate sheets if needed.

Chapter 4 Worksheet: Building Logic with Algorithms and Flowcharts

Welcome to the Chapter 4 Worksheet for “Coding & App Development for Beginners”! This worksheet is designed for high school students to reinforce the concepts of algorithms, flowcharts, pseudocode, and their application in Python programming. Complete each section to test your understanding and practice your skills. This worksheet includes a variety of activities: brief interactive tasks, hands-on tasks, coding exercises, and a mini-project. Let's dive in and think like coders!

Part 1: Quick Concept Match-Up Activity (20 points)

Instructions: Match each term or concept on the left with the correct description on the right by writing the corresponding letter (A-E) in the blank provided. Then, for each match, write one key word or short phrase (3-5 words) that summarizes why it's important in programming. Each question is worth 4 points (2 for the match, 2 for the key phrase).

- **Algorithm** ____ Key Phrase: _____
- **Flowchart** ____ Key Phrase: _____
- **Pseudocode** ____ Key Phrase: _____
- **Modulo Operator (%)** ____ Key Phrase: _____
- **Planning Before Coding** ____ Key Phrase: _____

Options: A. A visual diagram using shapes to map out a process B. A step-by-step set of instructions to solve a problem C. A plain-language draft of a program's logic D. A tool in Python to find the remainder after division E. The process of organizing logic before writing actual code

Part 2: Flowchart Drawing Activity (15 points)

Task: Draw a flowchart for deciding what to do after school based on two conditions: “Do I have homework?” and “Is the weather good?” Include at least three possible outcomes (e.g., “Study at home,” “Hang out with friends,” “Play sports outside”). Use the correct flowchart symbols (Oval for Start/End, Diamond for Decisions, Rectangle for Actions, Arrows for Flow).

- **Points Breakdown:**
 - Correct use of symbols: 5 points
 - Logical flow with at least two decisions: 5 points
 - Clear outcomes (at least three): 5 points

Space for Drawing or Description: (If you can't draw, describe the flowchart step by step with shapes and connections.)

Part 3: Pseudocode Writing Exercise (15 points)

Task: Write pseudocode for a “Simple Alarm Clock App” that asks the user for the current time and a wake-up time, then tells them how many hours they have left to sleep. Include a decision to check if the wake-up time is valid (e.g., not in the past).

- **Points Breakdown:**
 - Clear structure with BEGIN/END or similar markers: 5 points
 - Logical steps including input and calculation: 5 points
 - Decision point for validation: 5 points

Write Your Pseudocode Here:

Part 4: Coding Exercises (30 points)

Complete the following Python coding tasks based on concepts from Chapter 4. Each exercise is worth 10 points. Write the code in the space provided or on a separate sheet if needed. Test your code to ensure it works as expected.

- **Temperature Checker (10 points) Goal:** Write a program that asks the user for the current temperature (in Fahrenheit) and prints a message based on whether it’s “Hot” (above 80), “Comfortable” (60-80), or “Cold” (below 60).
 - **Points:** 5 for correct logic, 5 for working code. **Code Here:**
- **Simple Loop Counter (10 points) Goal:** Write a program that counts from 1 to 5 using a while loop, printing each number on a new line, and then prints “Done!” when finished.
 - **Points:** 5 for correct loop structure, 5 for correct output. **Code Here:**
- **Modified Vending Machine (10 points) Goal:** Modify the Vending Machine Simulator from the chapter to add a fourth item: “Water” for \$1.25. Update the menu and logic to handle this new choice.
 - **Points:** 5 for adding the new item, 5 for correct logic and output. **Code Here:**

Part 5: Mini-Project – Build a “Daily Planner Bot” (20 points)

Task: Create a small Python program that acts as a “Daily Planner Bot.” The program should:

- Ask the user for their name.
- Ask how many tasks they have for the day (1-5).
- Use a loop to collect each task as input (e.g., “Task 1: Math homework”).
- Finally, print a summary of all tasks with a motivational message (e.g., “Let’s crush it today, [Name]!”).

Steps to Follow:

- Plan your program with a short algorithm (list 3-5 steps).
- Write the Python code.
- **Points Breakdown:**
 - Algorithm plan: 5 points
 - Working code with input and loop: 10 points
 - Clear output with summary and message: 5 points

Algorithm Plan:

Python Code:

Bonus Challenge (Optional – 10 Extra Points)

Task: Enhance your “Daily Planner Bot” to include a decision: If the user has more than 3 tasks, print a warning

like "Whoa, busy day! Prioritize your tasks, [Name]!" Test your updated code. **Code or Description Here:**